



Université Hassan 1er

Faculté des Sciences et Techniques de Settat

Département Génie Informatique

Cycle d'Ingénieur en Informatique – 1^{ère} année

Rapport de Projet de Fin d'Année

Filière Génie Informatique

Conception et développement d'une application de gestion et de suivi des badges d'accès

Réalisé par :

Oussama Tarchi

Encadré par :

M. Hakim Bahir

Office National Des Aéroports (ONDA) – Aéroport de Dakhla

Service Technique Navigation

Année universitaire : 2025–2026

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude envers l'Office National Des Aéroports (ONDA) pour m'avoir accueilli au sein de l'aéroport de Dakhla et pour m'avoir donné l'occasion de réaliser ce projet dans un environnement professionnel concret.

Je remercie chaleureusement mon encadrant professionnel, M. Hakim Bahir, pour sa disponibilité, ses conseils avisés et le partage de son expérience tout au long de ce projet.

Je remercie vivement l'ensemble du personnel de l'aéroport de Dakhla pour son accueil, sa disponibilité et les explications fournies sur le fonctionnement du suivi des badges d'accès. Ces échanges m'ont permis de mieux comprendre les besoins opérationnels liés au renouvellement des badges et les enjeux de sûreté associés au contrôle des accès.

Enfin, je remercie les professeurs de la Faculté des Sciences et Techniques de Settat pour la qualité de la formation dispensée dans le Cycle d'Ingénieur en Informatique, ainsi que pour les bases théoriques et méthodologiques mobilisées au cours de ce projet.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un projet de fin d'année effectué au sein de l'Office National Des Aéroports (ONDA), à l'aéroport de Dakhla. Le projet porte sur la conception et le développement d'une application de bureau destinée à améliorer la gestion et le suivi des badges d'accès.

L'application développée en Java permet d'importer un fichier Excel contenant les informations relatives aux badges, d'identifier automatiquement les colonnes utiles et de calculer la date d'expiration à partir de la date de délivrance et de la durée prévue. Les badges sont ensuite classés selon leur état : valide, nécessitant une relance ou expiré. Une interface graphique Swing offre une visualisation centralisée, des filtres par état et un suivi actualisé des données.

Le système intègre également des alertes locales persistantes, un mécanisme de relance manuelle ou automatique par courrier électronique via SMTP, ainsi qu'un historique JSON permettant d'éviter l'envoi de rappels en double au cours d'une même journée. Cette solution contribue à réduire les opérations manuelles, à améliorer la traçabilité du suivi et à limiter le risque de maintien d'un accès avec un badge arrivé à expiration.

Mots-clés : Java, Swing, Apache POI, Excel, badges d'accès, expiration, notifications SMTP, ONDA, Dakhla.

Abstract

This report presents the work carried out during a final-year project at the National Airports Authority of Morocco (ONDA), Dakhla Airport. The project focuses on the design and development of a desktop application for managing and monitoring airport access badges.

The Java application imports an Excel workbook containing badge records, automatically identifies the relevant columns, and calculates the expiration date from the delivery date and the planned duration. Badges are classified as valid, requiring a reminder, or expired. A Swing graphical interface provides a centralized view of the records, status-based filters, and refreshed monitoring of the source data.

The solution also includes persistent local alerts, manual and automatic SMTP email reminders, and a JSON history mechanism that prevents duplicate reminders from being sent on the same day. The application therefore reduces manual follow-up work, improves traceability, and helps prevent access badges from remaining active after their expiration date.

Keywords : Java, Swing, Apache POI, Excel, access badges, expiration monitoring, SMTP notifications, ONDA, Dakhla.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Introduction générale	7
1 Présentation de l'organisme d'accueil	8
1.1 Introduction	8
1.2 L'Office National Des Aéroports (ONDA)	8
1.3 L'aéroport de Dakhla	8
1.4 Le Service Technique Navigation et cadre du projet	9
1.5 Conclusion	9
2 Contexte et cadrage du projet	10
2.1 Introduction	10
2.2 Analyse de l'existant	10
2.3 Problématique	11
2.4 Objectifs du projet	11
2.5 Principes fonctionnels retenus	11
2.6 Conclusion	11
3 Conception	12
3.1 Introduction	12
3.2 Spécifications fonctionnelles	12
3.2.1 Importation et lecture du fichier Excel	13
3.2.2 Calcul de la date d'expiration	14
3.2.3 Visualisation et filtrage des badges	14
3.2.4 Alertes locales et relances par courrier électronique	15
3.2.5 Modes manuel et automatique	15
3.2.6 Traçabilité et prévention des doublons	16
3.3 Diagramme de classes	16

3.4	Conception de l'interface utilisateur	17
3.5	Conception de la persistance locale	18
3.6	Choix technologiques	18
3.7	Contraintes et limites de conception	19
3.8	Conclusion	20
4	Réalisation	21
4.1	Introduction	21
4.2	Architecture générale de l'application	21
4.3	Structure du projet Maven	22
4.4	Réalisation du modèle métier	23
4.5	Parsing Excel avec Apache POI	23
4.6	Calcul et classification des échéances	24
4.7	Interface graphique Java Swing	24
4.8	Système de notifications locales	25
4.9	Envoi SMTP et génération des courriels HTML	26
4.10	Historique JSON et prévention des envois en double	27
4.11	Surveillance du fichier et relance automatique	28
4.12	Conclusion	28
5	Tests, validation et perspectives	29
5.1	Introduction	29
5.2	Objectifs des tests	29
5.3	Environnement de test	30
5.4	Tests de lecture du fichier Excel	30
5.5	Tests du calcul des échéances	31
5.6	Tests de l'interface graphique	31
5.7	Tests des notifications locales	33
5.8	Tests d'envoi des e-mails	34
5.9	Tests de l'historique	35
5.10	Validation globale de la solution	36
5.11	Limites de la solution	36
5.12	Perspectives d'amélioration	37
5.13	Conclusion	37
	Conclusion générale	38

Table des figures

2.1	Structure du fichier Excel exploité par l'application.	10
3.1	Diagramme de classes organisé de l'application ONDA Badges Dakhla .	17
4.1	Structure du projet	22
4.2	Interface principale de l'application	25
4.3	Exemple de notification envoyée par e-mail	27
4.4	Fenêtre d'historique de l'application	28
5.1	Test de l'interface principale	32
5.2	Test de l'interface principale	32
5.3	Test de l'interface principale	32
5.4	Affichage d'une notification locale pour les badges nécessitant une attention	33
5.5	fichier json	34
5.6	Test d'envoi d'une notification par e-mail	34
5.7	Test d'envoi d'une notification par e-mail	35
5.8	Tests de l'historique des emails	36

Introduction générale

Dans un environnement aéroportuaire, la maîtrise des accès constitue un élément essentiel pour assurer la sûreté des personnes, des infrastructures et des opérations. Les badges d'accès délivrés aux agents, aux prestataires et aux organismes partenaires sont accordés pour une durée déterminée. Leur expiration doit donc être suivie avec rigueur afin d'éviter les retards de renouvellement, les oublis administratifs ou toute situation pouvant entraîner un accès non autorisé.

Au sein de l'aéroport de Dakhla, les informations relatives aux badges sont regroupées dans un classeur Excel. La vérification manuelle des dates d'expiration, l'identification des badges expirés ou proches de l'échéance et l'envoi des relances deviennent répétitifs et exposés aux erreurs lorsque le nombre d'enregistrements augmente.

Le projet « ONDA Badges Dakhla » répond à ce besoin par le développement d'une application desktop en Java permettant de lire le fichier Excel, de calculer automatiquement les dates d'expiration, de visualiser l'état des badges et d'envoyer des notifications par courrier électronique. L'application intègre aussi un historique afin d'éviter les doublons et de garder une trace des alertes effectuées.

Ce rapport présente les étapes de réalisation du projet et s'articule autour de cinq chapitres :

1. **Présentation de l'organisme d'accueil** : présentation de l'ONDA, de l'aéroport de Dakhla, du service d'accueil et du cadre général du stage.
2. **Contexte général et analyse du besoin** : étude de la problématique, des objectifs du projet, des besoins fonctionnels et des contraintes à respecter.
3. **Conception de la solution** : description de l'architecture retenue, du modèle de données, des composants principaux et des choix de conception.
4. **Réalisation et mise en œuvre** : présentation de l'environnement de développement, des technologies utilisées, des interfaces et des fonctionnalités réalisées.
5. **Tests, validation et perspectives** : présentation des essais effectués, des résultats obtenus, des limites de la solution et des améliorations envisageables.

Enfin, une conclusion générale résume le travail réalisé, ses apports et les perspectives d'amélioration.

Chapitre 1

Présentation de l'organisme d'accueil

1.1 Introduction

Ce chapitre présente l'organisme dans lequel le projet a été réalisé, à savoir l'Office National Des Aéroports, ainsi que le site d'accueil : l'aéroport de Dakhla et son Service Technique Navigation. Il permet de situer l'application dans son contexte professionnel et de comprendre l'importance du suivi des badges d'accès dans un environnement aéroportuaire.

1.2 L'Office National Des Aéroports (ONDA)

L'Office National Des Aéroports est l'organisme marocain chargé de la gestion, de l'exploitation, de l'entretien et du développement des aéroports civils ouverts à la circulation aérienne publique. Dans ce cadre, l'ONDA contribue au fonctionnement des infrastructures aéroportuaires et à la coordination des services nécessaires à la sécurité et à la sûreté des opérations.

Les activités d'un aéroport reposent sur la collaboration de plusieurs catégories d'agents et de prestataires. La délivrance de badges temporaires ou permanents permet de contrôler l'accès aux zones autorisées. Le suivi de leur validité s'inscrit ainsi dans une démarche de maîtrise des risques et de traçabilité administrative.

1.3 L'aéroport de Dakhla

L'aéroport de Dakhla dessert la ville de Dakhla et accueille des activités aériennes nécessitant la coordination de services opérationnels, techniques, administratifs et de sûreté. Le contexte du site implique la circulation de personnels appartenant à des organismes différents, ce qui explique la présence de données telles que le nom du titulaire, son organisme, l'adresse électronique, la date de délivrance et la durée prévue du badge.

1.4 Le Service Technique Navigation et cadre du projet

Ce stage a été réalisé au sein du Service Technique Navigation de l'aéroport de Dakhla. Ce service intervient dans un environnement opérationnel où la rigueur, la traçabilité et le respect des procédures sont essentiels au bon fonctionnement des activités aéroportuaires.

Même si le service est principalement lié au suivi technique des équipements de navigation aérienne, le contexte de travail implique également la circulation de différents agents, prestataires et intervenants dans des zones réglementées. La gestion des badges d'accès constitue donc un élément important pour assurer le contrôle des autorisations et le respect des règles de sûreté.

Dans ce cadre, le projet *ONDA Badges Dakhla* a été proposé afin de faciliter le suivi des badges à partir du fichier Excel existant. L'application permet d'exploiter les données déjà utilisées par le service, de calculer automatiquement les dates d'expiration et de signaler les badges expirés ou proches de l'échéance.

1.5 Conclusion

L'ONDA et l'aéroport de Dakhla constituent un cadre opérationnel dans lequel le contrôle des autorisations d'accès doit être réalisé avec régularité. Le chapitre suivant précise le fonctionnement existant, la problématique retenue et les objectifs fonctionnels de l'application développée.

2.3 Problématique

La problématique du projet peut être formulée de la manière suivante :

Comment gérer efficacement et automatiser le suivi des dates d'expiration des badges du personnel de l'aéroport, simplifier le processus de renouvellement et envoyer des notifications électroniques afin de prévenir les risques de sécurité et les accès non autorisés, au moyen d'une application de bureau centralisée ?

2.4 Objectifs du projet

La solution vise principalement à :

- importer les données d'un fichier Excel au format `.xlsx` ;
- calculer automatiquement la date d'expiration ;
- distinguer les badges valides, proches de l'expiration et expirés ;
- proposer une interface Swing centralisée avec filtres et indicateurs visuels ;
- afficher des alertes locales ;
- envoyer des rappels HTML par SMTP en mode manuel ou automatique ;
- conserver un historique JSON pour éviter les relances répétées.

2.5 Principes fonctionnels retenus

Pour chaque badge, l'échéance est calculée selon la règle suivante :

$\{ \text{date d'expiration} = \text{date de délivrance} + \text{durée prévue en jours.} \}$

Le nombre de jours restants est ensuite comparé à un seuil configurable dans l'interface. Un badge dont la date est dépassée est classé comme expiré. Un badge dont l'échéance est comprise entre aujourd'hui et le seuil sélectionné nécessite une relance. Les autres badges sont considérés comme valides.

2.6 Conclusion

Le cadrage met en évidence le rôle de l'application comme outil intermédiaire entre le fichier Excel existant et les actions de suivi. Il conserve une source de données familière tout en automatisant le calcul des échéances, la détection des situations urgentes et la notification des personnes concernées.

Chapitre 3

Conception

3.1 Introduction

Après l'étude de l'existant et la définition de la problématique, cette partie présente la conception retenue pour l'application *ONDA Badges Dakhla*. L'objectif de cette conception est de transformer un besoin administratif réel en une solution logicielle claire, fiable et simple à utiliser par le service concerné.

La solution conçue est une application bureautique Java permettant de lire le fichier Excel utilisé pour le suivi des laissez-passer, de calculer automatiquement les dates d'expiration, de classer les badges selon leur niveau d'urgence et de déclencher des alertes locales ou des relances par courrier électronique. Le choix d'une application desktop se justifie par le contexte de travail : les données sont déjà centralisées dans un fichier Excel local, les agents utilisent un poste Windows, et le besoin principal est d'automatiser le contrôle sans imposer une infrastructure serveur ou une base de données.

La conception a donc été orientée vers trois principes : automatiser les tâches répétitives, conserver une interface lisible pour l'utilisateur, et assurer une traçabilité minimale des alertes et des courriels envoyés.

3.2 Spécifications fonctionnelles

Les spécifications fonctionnelles décrivent les services que l'application doit fournir à l'utilisateur final. Dans ce projet, l'utilisateur principal est l'agent chargé du suivi des badges ou toute personne du service habilitée à contrôler les échéances des laissez-passer.

Besoin	Description fonctionnelle
Importer les données	Charger un fichier Excel contenant la liste des badges et extraire automatiquement les informations nécessaires.
Calculer les échéances	Déterminer la date d'expiration à partir de la date de délivrance et de la durée prévue.
Visualiser les badges	Afficher les badges dans un tableau lisible, avec les informations essentielles et un statut visuel.
Filtrer les urgences	Permettre l'affichage des badges expirés, des badges à relancer ou de l'ensemble des badges.
Notifier localement	Afficher une alerte persistante lorsqu'un badge expiré ou proche de l'expiration est détecté.
Envoyer les relances	Générer et envoyer des e-mails HTML aux détenteurs des badges concernés.
Éviter les doublons	Mémoriser les alertes consultées et les e-mails déjà envoyés afin d'éviter les répétitions inutiles.

TABLE 3.1 – Synthèse des besoins fonctionnels de l'application

3.2.1 Importation et lecture du fichier Excel

Le fichier Excel constitue la source principale des données. L'application doit permettre à l'utilisateur de sélectionner ce fichier depuis l'interface graphique et de mémoriser le chemin choisi pour les utilisations suivantes. Le fichier attendu correspond à la liste des laissez-passer, contenant notamment le numéro du badge, le nom du détenteur, son e-mail, son organisme, la date de délivrance et la durée prévue.

La conception ne repose pas sur une position figée des colonnes uniquement. Même si le format standard place les en-têtes à la quatrième ligne du fichier, le service de lecture doit être capable de détecter dynamiquement la ligne d'en-tête dans les premières lignes du classeur.

Champ	Utilisation dans l'application
N° Badge	Identifiant principal du laissez-passer.
Nom et Prénom	Identification du détenteur du badge.
Email	Adresse utilisée pour l'envoi des relances automatiques.
Organisme	Société, organisme ou service auquel appartient le détenteur.
Délivré par	Information de suivi sur l'autorité ou l'agent de délivrance.
D-délivrance	Date de départ pour le calcul de l'échéance.
Durée prévue	Nombre de jours à ajouter à la date de délivrance.

TABLE 3.2 – Champs Excel exploités par la solution

3.2.2 Calcul de la date d'expiration

La date d'expiration n'est pas saisie manuellement par l'utilisateur dans l'application. Elle est calculée automatiquement afin de limiter les erreurs et d'assurer une règle uniforme pour tous les badges. La règle métier principale est la suivante :

$$\text{Date d'expiration} = \text{Date de délivrance} + \text{Durée prévue en jours}$$

Par exemple, si un badge est délivré le 18/07/2026 avec une durée prévue de 10 jours, l'application calcule une expiration au 28/07/2026. Cette règle permet de transformer des informations brutes issues du tableau Excel en indicateurs directement exploitables par l'agent.

Le calcul tient compte de plusieurs formats de dates, notamment les dates Excel natives, les formats numériques classiques et les mois écrits en français comme *juil*, *août* ou *déc*. La durée prévue est interprétée comme un nombre de jours : une valeur comme *15 JOURS* ou *15* donne donc le même résultat fonctionnel.

3.2.3 Visualisation et filtrage des badges

L'interface principale est conçue comme un tableau de bord. Elle regroupe les paramètres de configuration, le choix du fichier Excel, les options de relance et une zone de visualisation des badges. Le tableau affiche uniquement les informations nécessaires à la prise de décision : numéro du badge, nom et prénom, organisme, e-mail, date d'expiration et statut.

Trois vues de filtrage sont prévues :

- **Tous** : affiche l'ensemble des badges chargés depuis le fichier Excel.
- **À relancer** : affiche les badges dont l'expiration est comprise entre la date du jour et le délai de rappel sélectionné.
- **Expirés** : affiche les badges dont la date d'expiration est déjà dépassée.

Le délai de rappel est configurable par l'utilisateur. Les valeurs proposées sont 1, 3, 7, 14 et 30 jours. Ce choix permet d'adapter le suivi au niveau d'anticipation souhaité par le service.

Cas	Condition	Interprétation
Expiré	Jours restants < 0	Le badge n'est plus valide et nécessite une action immédiate.
À relancer	$0 \leq \text{jours restants} \leq \text{délai choisi}$	Le badge est encore valide, mais son renouvellement doit être anticipé.
Valide	Jours restants $> \text{délai choisi}$	Aucun traitement urgent n'est nécessaire.

TABLE 3.3 – Règles de classification des badges

3.2.4 Alertes locales et relances par courrier électronique

La solution distingue deux types d'alertes complémentaires.

Le premier type est l'alerte locale. Lorsqu'un badge expiré ou proche de l'expiration est détecté, l'application affiche une fenêtre d'alerte persistante dans l'environnement Windows. Cette alerte attire l'attention de l'utilisateur sans nécessiter l'ouverture permanente du fichier Excel. Pour éviter que la même alerte ne soit affichée à chaque rafraîchissement, les badges déjà consultés sont enregistrés dans le fichier local `seen_badges.json`.

Le second type est la relance par e-mail. L'application prépare un message HTML en français contenant les informations essentielles du badge : numéro, nom du détenteur, organisme, date d'expiration et date du jour. Le message est envoyé via un serveur SMTP configuré depuis l'interface.

3.2.5 Modes manuel et automatique

Deux modes d'exécution ont été prévus.

En mode manuel, l'utilisateur clique sur le bouton *LANCER* après avoir vérifié le fichier, le délai de rappel et les paramètres SMTP. Ce mode donne un contrôle direct à

l'agent et convient particulièrement aux tests, aux envois ponctuels ou aux situations nécessitant une validation humaine.

En mode automatique, l'application planifie l'envoi quotidien des relances à 09h00. Ce mode réduit le risque d'oubli et transforme le suivi des badges en un processus régulier. Le bouton d'envoi manuel est désactivé lorsque ce mode est actif, afin de rendre l'état de l'application plus clair pour l'utilisateur.

En parallèle, un mécanisme de surveillance vérifie périodiquement la date de modification du fichier Excel. Si le fichier est modifié, les données sont rechargées et les alertes locales sont recalculées.

3.2.6 Traçabilité et prévention des doublons

La traçabilité est un point important dans une application de relance. Sans mécanisme de suivi, un même détenteur pourrait recevoir plusieurs fois le même message dans la même journée, ou une même alerte pourrait réapparaître à chaque ouverture.

La conception prévoit donc deux fichiers d'historique :

- `seen_badges.json` pour mémoriser les badges déjà affichés dans les alertes locales ;
- `sent_emails_history.json` pour mémoriser les relances e-mail déjà envoyées.

L'identification d'une alerte locale repose sur une clé composée du numéro de badge et de sa date d'expiration. Ainsi, si un badge est renouvelé et que sa date d'expiration change, il peut de nouveau être signalé. Pour les e-mails, le système compare le numéro du badge, la date d'expiration et la date d'envoi afin d'empêcher les doublons le même jour.

3.3 Diagramme de classes

Le diagramme de classes présente les principales classes de l'application ONDA Badges Dakhla ainsi que leurs responsabilités et leurs relations. L'organisation retenue sépare l'interface graphique, le contrôle, les services métier et le modèle de données.

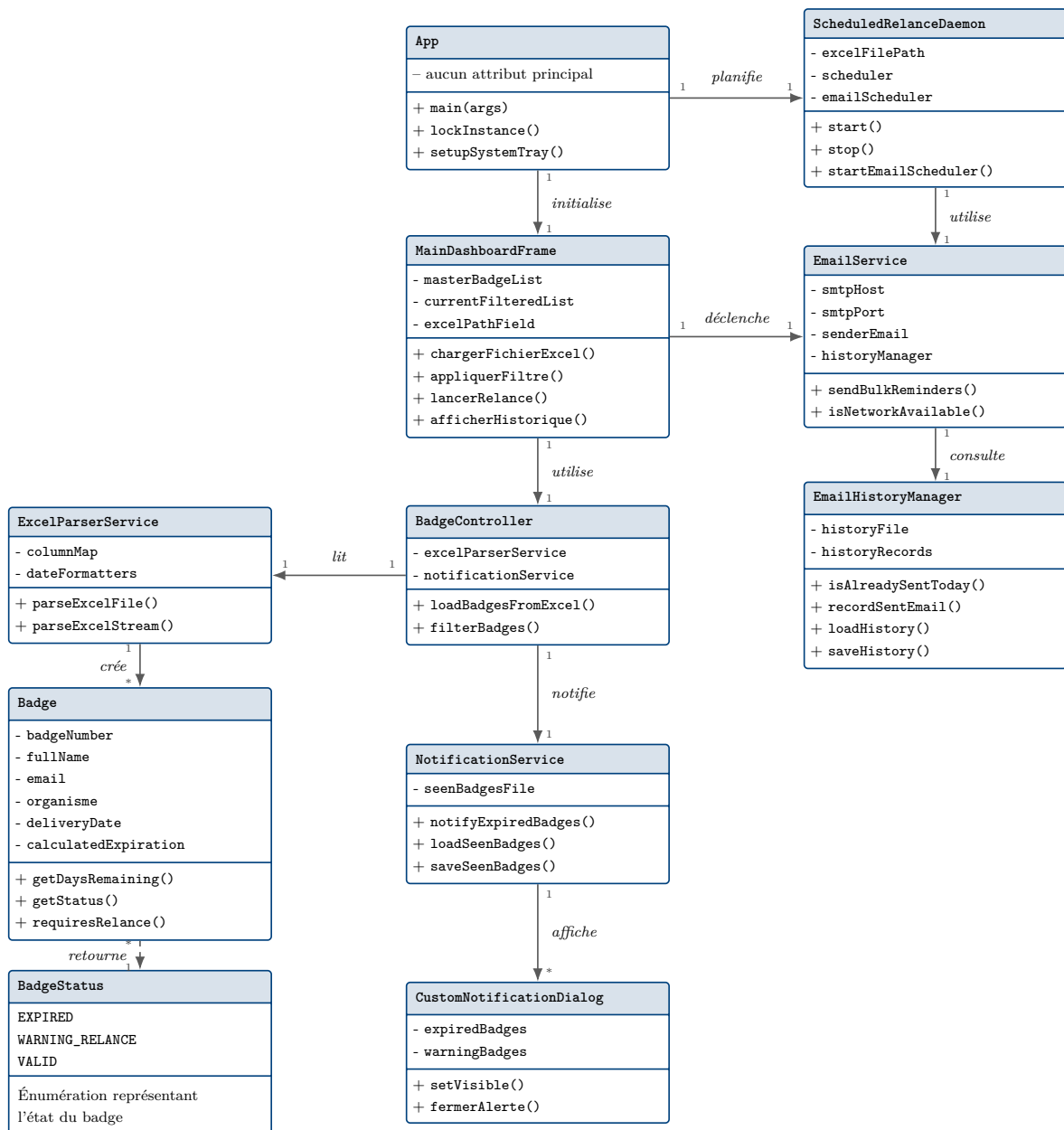


FIGURE 3.1 – Diagramme de classes organisé de l'application ONDA Badges Dakhla

3.4 Conception de l'interface utilisateur

L'interface a été pensée pour un usage administratif quotidien : elle doit être directe, lisible et suffisamment dense pour permettre un contrôle rapide des données. La fenêtre principale est divisée en plusieurs zones.

Le bandeau supérieur affiche l'identité de l'application avec le logo ONDA, le titre *ONDA Badges Dakhla – Application de Relance Automatique* et un bouton *Historique*. Ce bouton ouvre un journal séparé contenant deux onglets : les badges déjà vus et les e-mails envoyés.

Le panneau de configuration SMTP regroupe les paramètres nécessaires à l'envoi des courriels : serveur, port, adresse expéditeur et mot de passe. Le panneau de source d'entrée permet de sélectionner le fichier Excel. Le panneau *Configuration & Exécution* contient le délai de rappel, le choix du mode manuel ou automatique, ainsi que le bouton de lancement.

Enfin, la zone de visualisation présente les badges sous forme de tableau. Les lignes sont colorées selon leur état : rouge pour les badges expirés, jaune pour les badges à relancer et vert pour les badges valides. Cette représentation permet à l'utilisateur d'identifier immédiatement les priorités.

3.5 Conception de la persistance locale

La solution ne nécessite pas de base de données. Les données opérationnelles proviennent du fichier Excel, tandis que les préférences et les historiques sont stockés localement.

Fichier	Rôle
<code>application.properties</code>	Contient les valeurs par défaut de l'application, notamment le nom de l'application et le chemin Excel par défaut.
<code>config.properties</code>	Conserve les paramètres choisis par l'utilisateur : chemin du fichier Excel, serveur SMTP, port, e-mail expéditeur et mot de passe.
<code>seen_badges.json</code>	Mémorise les alertes locales déjà consultées afin de ne pas afficher plusieurs fois la même alerte.
<code>sent_emails_history.json</code>	Conserve l'historique des relances envoyées et permet d'éviter un nouvel envoi identique le même jour.

TABLE 3.4 – Fichiers de configuration et de persistance locale

3.6 Choix technologiques

Les choix technologiques ont été faits en fonction des contraintes du projet : application locale, manipulation d'un fichier Excel, interface simple et envoi d'e-mails.

Technologie	Justification
Java 17	Version moderne et stable du langage Java, adaptée aux applications desktop et à l'organisation en classes métiers.
Swing / AWT	Bibliothèques natives Java permettant de créer une interface bureautique, des boîtes de dialogue, une table et une icône système.
FlatLaf	Améliore l'apparence de Swing avec un rendu plus moderne et plus confortable pour l'utilisateur.
Maven	Structure le projet, gère les dépendances et permet de produire un fichier JAR exécutable avec ses dépendances.
Apache POI	Bibliothèque spécialisée dans la lecture et la manipulation des fichiers Excel au format <code>.xlsx</code> .
JavaMail	Fournit les mécanismes nécessaires à l'envoi de courriels via SMTP et STARTTLS.
JSON local	Permet une persistance simple des alertes vues et des e-mails envoyés sans installer de base de données.
JUnit 5	Permet de valider les règles sensibles comme la lecture Excel, le calcul des dates et la persistance des alertes.

TABLE 3.5 – Choix technologiques de la solution

3.7 Contraintes et limites de conception

Certaines limites ont été acceptées volontairement pour garder une solution réaliste dans le cadre du stage. L'application fonctionne localement et ne gère pas une base de données partagée entre plusieurs postes. Elle ne remplace donc pas un système d'information centralisé, mais elle automatise efficacement le suivi à partir du fichier Excel existant.

La fiabilité des résultats dépend également de la qualité du fichier source. Si des colonnes essentielles sont absentes, si les dates ne sont pas lisibles ou si les adresses e-mail sont incorrectes, l'application ne peut pas garantir un traitement complet. Pour réduire ce risque, la conception prévoit une détection dynamique des en-têtes, plusieurs formats de dates et l'ignorance contrôlée des lignes incomplètes.

Enfin, la configuration SMTP est stockée localement pour simplifier l'utilisation. Dans une version future, cette partie pourrait être renforcée par un chiffrement du

mot de passe, une authentification utilisateur ou une intégration avec une messagerie institutionnelle.

3.8 Conclusion

La conception proposée répond à la problématique initiale en organisant la solution autour d'un tableau de bord local, d'un moteur de lecture Excel, d'un calcul automatique des échéances et d'un système de notifications. Elle respecte le fonctionnement existant du service tout en y ajoutant une couche d'automatisation et de traçabilité.

Cette conception servira de base au chapitre suivant, consacré à la réalisation technique de l'application. Le chapitre de réalisation détaillera la manière dont les classes, les services, l'interface Swing, l'envoi SMTP et les fichiers JSON ont été effectivement implémentés dans le projet.

Chapitre 4

Réalisation

4.1 Introduction

Après la phase d'analyse et de conception, cette partie présente la réalisation concrète de l'application « ONDA Badges Dakhla ». L'objectif de ce chapitre est de décrire le passage du modèle conçu vers une application fonctionnelle, capable de lire un fichier Excel, d'analyser les informations des badges, de calculer leurs échéances, d'afficher leur état dans une interface graphique et d'envoyer des notifications aux personnes concernées.

Ce chapitre détaille donc les principaux éléments développés : la structure du projet Maven, le modèle métier, le parsing du fichier Excel, le calcul des dates d'expiration, l'interface graphique, les notifications locales, l'envoi des courriels, l'historique des actions et le mécanisme de relance automatique.

4.2 Architecture générale de l'application

L'application repose sur une architecture locale en couches. Cette organisation permet de séparer les responsabilités entre l'interface graphique, la logique métier, les services techniques et les fichiers de configuration ou d'historique.

La couche présentation est principalement représentée par la fenêtre principale `MainDashboardFrame`. Elle permet à l'utilisateur de configurer les paramètres SMTP, de choisir le fichier Excel, de lancer l'analyse des badges, de filtrer les résultats et de consulter l'historique. Elle est complétée par `CustomNotificationDialog`, qui affiche les alertes locales.

La couche contrôle est assurée par `BadgeController`. Cette classe joue le rôle d'intermédiaire entre l'interface et les services. Elle déclenche le chargement des badges, applique les filtres et transmet les résultats à l'interface.

La couche service regroupe les classes responsables des traitements techniques : `ExcelParserService` pour la lecture du fichier Excel, `NotificationService` pour les alertes locales, `EmailService` pour l'envoi des e-mails, `EmailHistoryManager` pour l'historique des courriels et `ScheduledRelanceDaemon` pour les traitements automatiques.

Couche	Rôle
Présentation	Affichage de l'interface, saisie des paramètres, visualisation des badges et consultation de l'historique.
Contrôle	Coordination entre l'interface graphique et les services métier.
Services	Lecture Excel, calculs, notifications, envoi des e-mails et gestion des historiques.
Stockage local	Conservation des paramètres, des badges déjà signalés et des e-mails déjà envoyés.

TABLE 4.1 – Organisation générale de l'application

4.3 Structure du projet Maven

Le projet a été organisé sous forme d'une application Java Maven. Maven facilite la gestion des dépendances, la compilation du projet et la génération de l'application exécutable. La structure retenue suit l'organisation standard d'un projet Java, avec un dossier `src/main/java` pour le code source et un dossier `src/main/resources` pour les ressources.

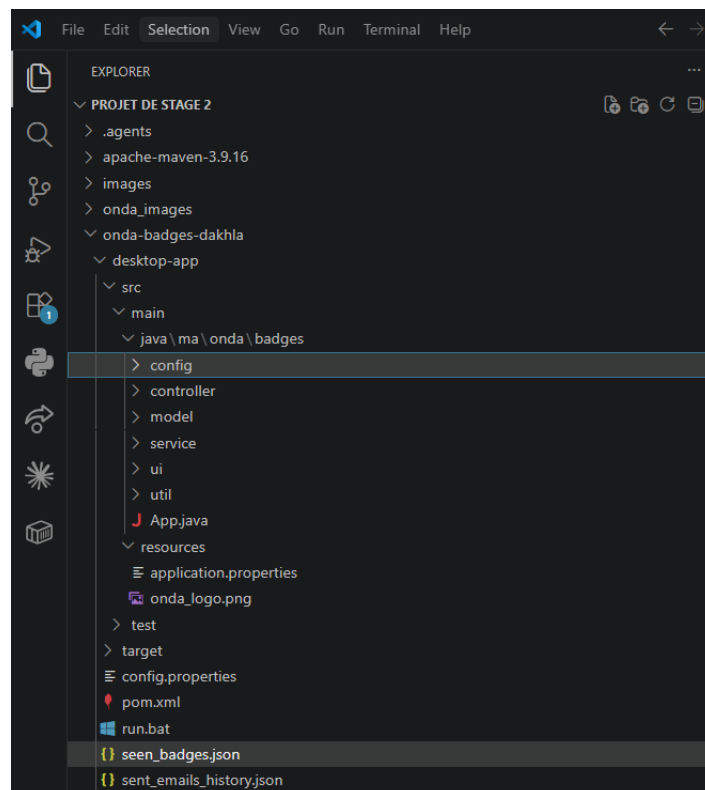


FIGURE 4.1 – Structure du projet

Le fichier `pom.xml` contient les dépendances nécessaires au fonctionnement de l'application. Les principales bibliothèques utilisées sont Apache POI pour la lecture des fichiers Excel, JavaMail pour l'envoi des courriels, FlatLaf pour améliorer l'apparence de l'interface Swing, ainsi que JUnit pour les tests.

4.4 Réalisation du modèle métier

Le modèle métier principal de l'application est représenté par la classe `Badge`. Chaque objet `Badge` correspond à une ligne utile extraite du fichier Excel. Il regroupe les informations nécessaires au suivi du badge : numéro, nom complet, adresse e-mail, organisme, date de délivrance, durée prévue et date d'expiration calculée.

La classe `Badge` ne se limite pas au stockage des données. Elle contient également des méthodes permettant de calculer le nombre de jours restants avant l'expiration et de déterminer l'état du badge. Cette approche permet de garder la logique liée au badge proche de l'objet concerné.

Élément	Description
<code>badgeNumber</code>	Numéro unique du badge.
<code>fullName</code>	Nom et prénom du détenteur.
<code>email</code>	Adresse utilisée pour l'envoi des notifications.
<code>organisme</code>	Organisme ou société du détenteur.
<code>deliveryDate</code>	Date de délivrance du badge.
<code>intendedDuration</code>	Durée prévue de validité.
<code>calculatedExpiration</code>	Date d'expiration calculée automatiquement.

TABLE 4.2 – Attributs principaux de la classe `Badge`

L'état du badge est représenté par l'énumération `BadgeStatus`. Trois états sont utilisés dans l'application : badge expiré, badge à relancer et badge valide. Cette classification facilite l'affichage coloré dans le tableau de bord et le filtrage des résultats.

4.5 Parsing Excel avec Apache POI

La lecture du fichier Excel est assurée par la classe `ExcelParserService`. Le choix d'Apache POI permet de manipuler les fichiers `.xlsx` directement depuis Java, sans dépendre de Microsoft Excel.

Le fichier utilisé par le service contient les informations des badges sous forme de tableau. L'application parcourt le classeur, identifie les colonnes utiles, lit les lignes de données puis transforme chaque ligne en objet **Badge**. Les colonnes prises en charge sont notamment le numéro du badge, le nom et prénom, l'e-mail, l'organisme, la date de délivrance et la durée prévue.

Une attention particulière a été portée à la détection des en-têtes. Le service analyse les premières lignes du fichier afin de retrouver automatiquement les colonnes, même si le tableau ne commence pas exactement à la première ligne. Ce choix rend l'application plus souple face au format réel du fichier Excel utilisé au sein du service.

Le parsing tient également compte des dates et des durées. Les dates peuvent être lues depuis des cellules Excel ou depuis du texte. La durée prévue est ensuite interprétée afin de calculer la date d'expiration du badge.

4.6 Calcul et classification des échéances

Une fois les données extraites, l'application calcule automatiquement la date d'expiration de chaque badge. Le principe utilisé est simple : la date de délivrance est additionnée à la durée prévue indiquée dans le fichier Excel.

$$\text{Date d'expiration} = \text{Date de délivrance} + \text{Durée prévue}$$

Après ce calcul, l'application compare la date d'expiration avec la date du jour. Cette comparaison permet de classer les badges en trois catégories :

- **Expiré** : la date d'expiration est déjà dépassée.
- **À relancer** : le badge est encore valide, mais son expiration est proche selon le délai choisi par l'utilisateur.
- **Valide** : le badge n'est pas encore concerné par une relance.

Le délai de relance est configurable depuis l'interface. L'utilisateur peut choisir un seuil comme 1 jour, 3 jours, 7 jours, 14 jours ou 30 jours. Cette souplesse permet d'adapter le fonctionnement de l'application selon les besoins du service.

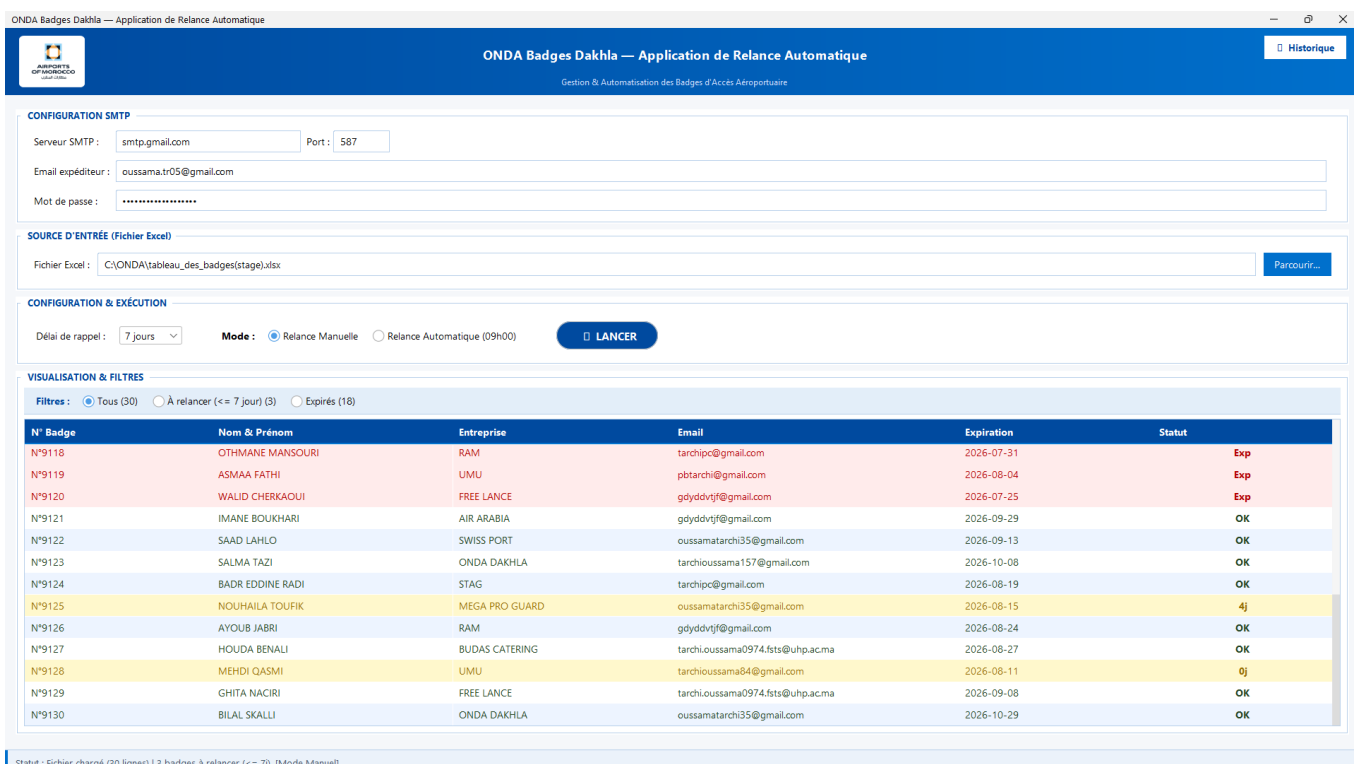
4.7 Interface graphique Java Swing

L'interface graphique a été développée avec Java Swing. Ce choix permet de créer une application de bureau légère, exécutable localement sur le poste de travail du service. L'apparence de l'interface a été améliorée à l'aide de FlatLaf, afin d'obtenir un rendu plus moderne et plus lisible.

La fenêtre principale `MainDashboardFrame` regroupe les fonctionnalités essentielles dans plusieurs panneaux :

- un panneau de configuration SMTP ;
- un panneau de sélection du fichier Excel ;
- un panneau de configuration du délai de relance et du mode d'exécution ;
- un tableau de visualisation des badges ;
- des filtres permettant d'afficher tous les badges, les badges expirés ou les badges à relancer ;
- un bouton d'accès à l'historique.

Le tableau de bord affiche les badges avec des couleurs différentes selon leur état. Les badges expirés sont mis en évidence, les badges proches de l'expiration sont signalés, et les badges valides sont affichés comme étant en règle. Cette présentation permet à l'utilisateur d'identifier rapidement les cas prioritaires.



ONDA Badges Dakhla — Application de Relance Automatique

Gestion & Automatisation des Badges d'Accès Aéroportuaire

CONFIGURATION SMTP

Serveur SMTP : smtp.gmail.com Port : 587

Email expéditeur : oussama.tr05@gmail.com

Mot de passe : *****

SOURCE D'ENTRÉE (Fichier Excel)

Fichier Excel : C:\ONDA\tablage_des_badges(stage).xlsx

CONFIGURATION & EXÉCUTION

Délai de rappel : 7 jours

Mode : ☒ Relance Manuelle ☐ Relance Automatique (09h00)

VISUALISATION & FILTRES

Filtres : ☒ Tous (30) ☐ À relancer (<= 7 jour) (3) ☐ Expirés (18)

N° Badge	Nom & Prénom	Entreprise	Email	Expiration	Statut
N°9118	OTHMANE MANSOURI	RAM	tarchipc@gmail.com	2026-07-31	Exp
N°9119	ASMAA FATHI	UMU	pbtarchi@gmail.com	2026-08-04	Exp
N°9120	WALID CHERKAOUI	FREE LANCE	gdyddvtj@gmail.com	2026-07-25	Exp
N°9121	IMANE BOUKHARI	AIR ARABIA	gdyddvtj@gmail.com	2026-09-29	OK
N°9122	SAAD LAHLO	SWISS PORT	oussamatarchi35@gmail.com	2026-09-13	OK
N°9123	SALMA TAZI	ONDA DAKHLA	tarchioussama157@gmail.com	2026-10-08	OK
N°9124	BADR EDDINE RADI	STAG	tarchipc@gmail.com	2026-08-19	OK
N°9125	NOUHAILA TOUFIK	MEGA PRO GUARD	oussamatarchi35@gmail.com	2026-08-15	4j
N°9126	AYOUB JABRI	RAM	gdyddvtj@gmail.com	2026-08-24	OK
N°9127	HOUDA BENALI	BUDAS CATERING	tarchi.oussama0974.fsts@uhp.ac.ma	2026-08-27	OK
N°9128	MEHDI QASMI	UMU	tarchioussama84@gmail.com	2026-08-11	0j
N°9129	GHITA NACIRI	FREE LANCE	tarchi.oussama0974.fsts@uhp.ac.ma	2026-09-08	OK
N°9130	BILAL SKALLI	ONDA DAKHLA	oussamatarchi35@gmail.com	2026-10-29	OK

Statut : Fichier chargé (30 lignes) | 3 badges à relancer (<= 7j). [Mode Manuel]

FIGURE 4.2 – Interface principale de l'application

4.8 Système de notifications locales

En plus de l'affichage dans le tableau, l'application intègre un système de notifications locales. Lorsqu'un badge expiré ou proche de l'expiration est détecté, une boîte d'alerte

peut être affichée à l'utilisateur. Cette fonctionnalité permet d'attirer l'attention du service sans nécessiter une vérification permanente du tableau.

La classe `NotificationService` est responsable de cette partie. Elle analyse la liste des badges et détermine ceux qui nécessitent une alerte. Pour éviter d'afficher plusieurs fois la même notification, les badges déjà signalés sont enregistrés dans le fichier `seen_badges.json`.

La fenêtre d'alerte est réalisée avec la classe `CustomNotificationDialog`. Elle présente les badges concernés et permet à l'utilisateur de fermer l'alerte après consultation.

4.9 Envoi SMTP et génération des courriels HTML

L'une des fonctionnalités principales de l'application est l'envoi automatique de courriels aux détenteurs de badges concernés. Cette partie est assurée par la classe `EmailService`, qui utilise le protocole SMTP avec authentification.

L'utilisateur peut configurer depuis l'interface le serveur SMTP, le port, l'adresse de l'expéditeur et le mot de passe. Dans le cas testé, le serveur Gmail peut être utilisé avec le port 587 et le protocole STARTTLS.

Les e-mails générés sont au format HTML afin d'obtenir un message clair et professionnel. Le contenu du message indique le nom du détenteur, le numéro du badge, l'organisme, la date d'expiration et la date du jour. Le message précise également si le badge est expiré ou s'il nécessite une relance avant expiration.

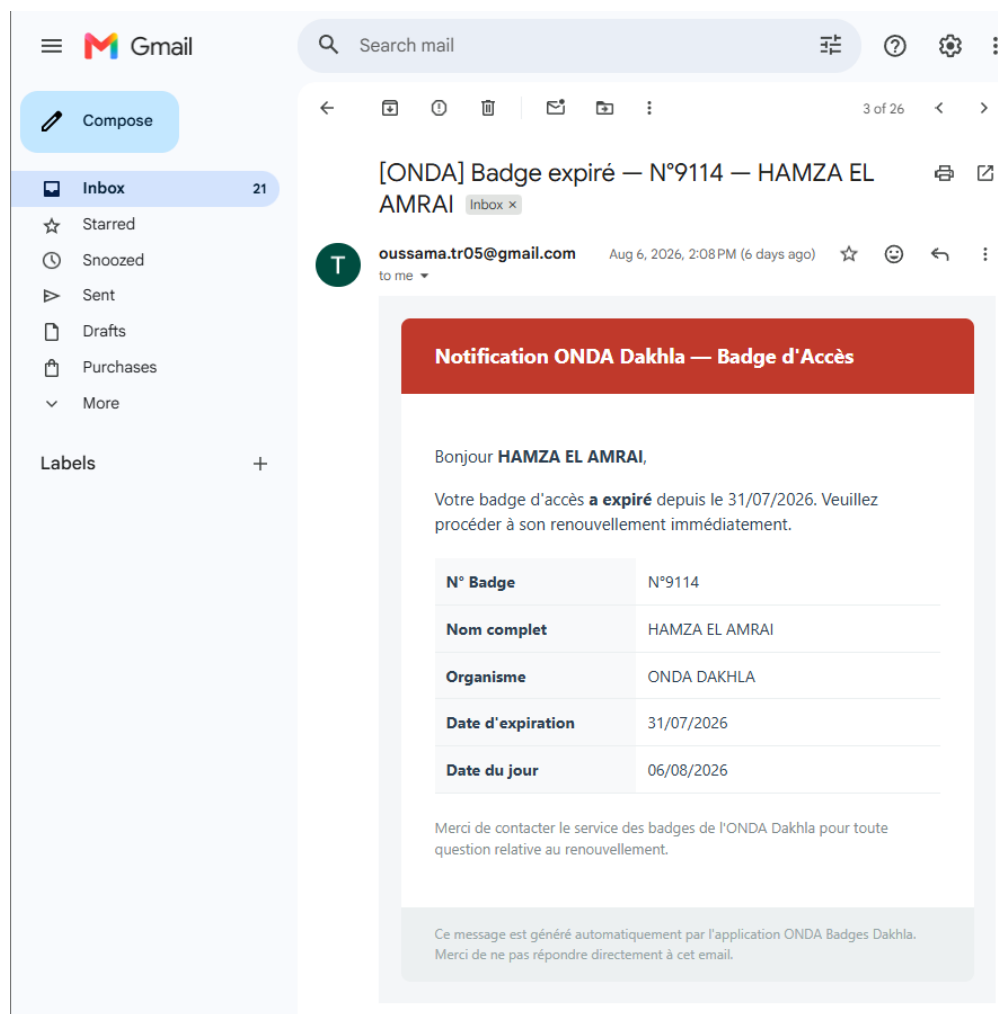


FIGURE 4.3 – Exemple de notification envoyée par e-mail

Avant l'envoi, l'application vérifie la disponibilité du réseau afin d'éviter les échecs immédiats liés à l'absence de connexion. Cette vérification améliore la fiabilité de l'opération et permet de mieux informer l'utilisateur en cas de problème.

4.10 Historique JSON et prévention des envois en double

Afin d'éviter l'envoi répété de plusieurs courriels au même détenteur pendant la même journée, l'application conserve un historique local des relances effectuées. Les e-mails envoyés sont enregistrés dans le fichier `sent_emails_history.json`, tandis que les badges déjà signalés sont mémorisés dans `seen_badges.json`.

Avant chaque nouvel envoi, l'application vérifie si une relance similaire a déjà été enregistrée pour le même badge et la même date d'expiration. Ce mécanisme permet d'éviter les doublons et de garder une trace des actions réalisées.

Journal & Historique ONDA

Journal des activités — Badges & E-mails

☐ Badges Vus ☐ Emails Envoyés

19 badge(s) enregistré(s) dans seen_badges.json

#	N° Badge	Date Vue
1	NÂ°9117	2026-07-26
2	NÂ°9128	2026-08-11
3	NÂ°9120	2026-07-25
4	NÂ°9103	2026-06-25
5	NÂ°9107	2026-08-01
6	NÂ°9113	2026-07-31
7	NÂ°9109	2026-07-30
8	NÂ°9111	2026-07-31
9	NÂ°9101	2026-07-27
10	NÂ°9114	2026-07-31
11	NÂ°9116	2026-07-28
12	NÂ°9119	2026-08-04
13	NÂ°9115	2026-07-28

FIGURE 4.4 – Fenêtre d'historique de l'application

La fenêtre d'historique, accessible depuis le bouton « Historique », permet au service de consulter les badges déjà vus et les e-mails envoyés. Elle renforce ainsi la traçabilité de l'application sans nécessiter une base de données externe.

4.11 Surveillance du fichier et relance automatique

La classe `ScheduledRelanceDaemon` assure les traitements automatiques en arrière-plan. Elle permet à l'application de surveiller le fichier Excel et de déclencher les relances sans intervention permanente de l'utilisateur.

4.12 Conclusion

Ce chapitre a présenté la réalisation technique de l'application *ONDA Badges Dakhla*. La solution permet de lire le fichier Excel, de calculer les échéances, de classer les badges et d'envoyer les notifications nécessaires. L'historique local et la prévention des doublons renforcent la fiabilité du suivi. Le chapitre suivant sera consacré aux tests, à la validation et aux perspectives d'amélioration.

Chapitre 5

Tests, validation et perspectives

5.1 Introduction

Après la réalisation de l'application *ONDA Badges Dakhla*, une phase de test a été nécessaire afin de vérifier le bon fonctionnement des principales fonctionnalités développées. Cette étape permet de s'assurer que l'application répond aux besoins définis au départ : lecture du fichier Excel, calcul correct des échéances, affichage des badges selon leur état, génération des alertes et envoi des relances par courrier électronique.

Les tests réalisés ont porté à la fois sur les traitements internes de l'application et sur son utilisation à travers l'interface graphique. L'objectif n'était pas seulement de vérifier que chaque fonctionnalité fonctionne séparément, mais aussi de valider le comportement global de la solution dans un cas proche de l'utilisation réelle au sein du service.

5.2 Objectifs des tests

La phase de test avait pour but de vérifier plusieurs points essentiels :

- la bonne lecture du fichier Excel contenant la liste des badges ;
- l'extraction correcte des informations nécessaires ;
- le calcul automatique des dates d'expiration ;
- la classification des badges selon leur état ;
- le fonctionnement des filtres du tableau de bord ;
- l'affichage des alertes locales ;
- l'envoi des e-mails de relance ;
- la mémorisation des badges déjà signalés et des e-mails déjà envoyés.

Ces vérifications permettent de confirmer que l'application réduit les traitements manuels et améliore le suivi des échéances des badges d'accès.

5.3 Environnement de test

Les tests ont été réalisés dans un environnement local, similaire à celui prévu pour l'utilisation de l'application. Le projet a été exécuté sur un poste Windows à l'aide d'un environnement Java et de Maven.

Élément	Description
Système d'exploitation	Windows
Langage	Java
Gestion du projet	Maven
Interface graphique	Java Swing avec FlatLaf
Source de données	Fichier Excel des badges
Envoi des e-mails	Serveur SMTP configuré depuis l'application
Stockage local	Fichiers JSON et fichier de configuration

TABLE 5.1 – Environnement utilisé pour les tests

5.4 Tests de lecture du fichier Excel

Le premier test a consisté à charger le fichier Excel contenant les informations des badges. L'application devait reconnaître les colonnes utiles, parcourir les lignes du tableau et transformer chaque enregistrement en objet exploitable.

Les données testées comprenaient notamment le numéro du badge, le nom et prénom, l'adresse e-mail, l'organisme, la date de délivrance et la durée prévue. Le résultat attendu était l'affichage de l'ensemble des badges dans le tableau de bord.

Test	Résultat attendu	État
Chargement du fichier Excel	Les badges sont lus et affichés dans le tableau	Validé
Détection des colonnes	Les champs utiles sont reconnus automatiquement	Validé
Lecture des dates	Les dates de délivrance sont interprétées correctement	Validé
Lecture des durées	La durée prévue est convertie en nombre de jours	Validé

TABLE 5.2 – Tests de lecture du fichier Excel

5.5 Tests du calcul des échéances

Après l'importation des données, l'application calcule la date d'expiration de chaque badge à partir de la date de délivrance et de la durée prévue. Ce calcul est essentiel, car il conditionne l'état final du badge.

Les tests ont permis de vérifier que les badges sont correctement classés en trois catégories : expirés, à relancer ou valides.

Cas testé	Condition	Résultat
Badge expiré	Date d'expiration dépassée	Badge marqué comme expiré
Badge à relancer	Expiration proche du délai choisi	Badge marqué à relancer
Badge valide	Expiration éloignée	Badge marqué valide

TABLE 5.3 – Tests de classification des badges

Ces tests confirment que le calcul automatique permet de remplacer la vérification manuelle des dates et de limiter les risques d'erreur.

5.6 Tests de l'interface graphique

L'interface graphique a été testée afin de vérifier sa lisibilité et la bonne interaction entre les différents composants. Les principaux éléments vérifiés sont le choix du fichier

Excel, le tableau des badges, les filtres et le bouton de lancement.

SOURCE D'ENTRÉE (Fichier Excel)

Fichier Excel : C:\ONDA\tableau_des_badges(stage).xlsx Parcourir...

CONFIGURATION & EXÉCUTION

Délai de rappel : 7 jours Mode : ☒ Relance Manuelle ☐ Relance Automatique (09h00) LANCER

VISUALISATION & FILTRES

Filtres : ☐ Tous (30) ☐ À relancer (<= 7 jours) (3) ☐ Expirés (19)

N° Badge	Nom & Prénom	Entreprise	Email	Expiration	Statut
N°9103	SAADEDINE BOURHIM	DOHP	tarchioussama4@gmail.com	2026-06-25	Exp
N°9104	MOHAMMED EL MHAYOUI	MEGA PRO GUARD	pbtarchi@gmail.com	2026-07-26	Exp
N°9108	ALI ZOULNE	RAM	tarchipc@gmail.com	2026-07-20	Exp
N°9101	IDOUCH MARYAM	BUDAS CATERING	tarchioussama0974.fsts@uhp.ac.ma	2026-07-27	Exp
N°9107	TAHA RAEEL	UMU	xgddgffwte@gmail.com	2026-08-01	Exp
N°9100	OUSSAMA TARCHI	MEGA PRO GUARD	gdyddvtf@gmail.com	2026-07-28	Exp
N°9105	MAROUAN BENMOUINA	RAM	gdyddvtf@gmail.com	2026-08-30	OK
N°9102	HAIJAR BADIL	UMU	earstzhdzgr@gmail.com	2026-08-14	2j
N°9109	YOUSSEF ALAMI	ONDA DAKHLA	tarchioussama0974.fsts@uhp.ac.ma	2026-07-30	Exp
N°9110	KHALID BENNANI	FREE LANCE	pbtarchi@gmail.com	2026-08-01	Exp
N°9111	ZINEB CHRAIBI	AIR ARABIA	tarchioussama4@gmail.com	2026-07-31	Exp
N°9112	KARIM SAAD	ROYAL AIR MAROC	tarchioussama0974.fsts@uhp.ac.ma	2026-08-02	Exp
N°9113	FATIMA ZAHRA NADIR	SWISS PORT	oussamatarchi35@gmail.com	2026-07-31	Exp

FIGURE 5.1 – Test de l'interface principale

SOURCE D'ENTRÉE (Fichier Excel)

Fichier Excel : C:\ONDA\tableau_des_badges(stage).xlsx Parcourir...

CONFIGURATION & EXÉCUTION

Délai de rappel : 7 jours Mode : ☒ Relance Manuelle ☐ Relance Automatique (09h00) LANCER

VISUALISATION & FILTRES

Filtres : ☐ Tous (30) ☒ À relancer (<= 7 jours) (3) ☐ Expirés (19)

N° Badge	Nom & Prénom	Entreprise	Email	Expiration	Statut
N°9102	HAIJAR BADIL	UMU	earstzhdzgr@gmail.com	2026-08-14	2j
N°9124	BADR EDDINE RADI	STAG	tarchipc@gmail.com	2026-08-19	7j
N°9125	NOUHAILA TOURIK	MEGA PRO GUARD	oussamatarchi35@gmail.com	2026-08-15	3j

FIGURE 5.2 – Test de l'interface principale

SOURCE D'ENTRÉE (Fichier Excel)

Fichier Excel : C:\ONDA\tableau_des_badges(stage).xlsx Parcourir...

CONFIGURATION & EXÉCUTION

Délai de rappel : 7 jours Mode : ☒ Relance Manuelle ☐ Relance Automatique (09h00) LANCER

VISUALISATION & FILTRES

Filtres : ☐ Tous (30) ☐ À relancer (<= 7 jours) (3) ☒ Expirés (19)

N° Badge	Nom & Prénom	Entreprise	Email	Expiration	Statut
N°9103	SAADEDINE BOURHIM	DOHP	tarchioussama4@gmail.com	2026-06-25	Exp
N°9104	MOHAMMED EL MHAYOUI	MEGA PRO GUARD	pbtarchi@gmail.com	2026-07-26	Exp
N°9108	ALI ZOULNE	RAM	tarchipc@gmail.com	2026-07-20	Exp
N°9101	IDOUCH MARYAM	BUDAS CATERING	tarchioussama0974.fsts@uhp.ac.ma	2026-07-27	Exp
N°9107	TAHA RAEEL	UMU	xgddgffwte@gmail.com	2026-08-01	Exp
N°9100	OUSSAMA TARCHI	MEGA PRO GUARD	gdyddvtf@gmail.com	2026-07-28	Exp
N°9109	YOUSSEF ALAMI	ONDA DAKHLA	tarchioussama0974.fsts@uhp.ac.ma	2026-07-30	Exp
N°9110	KHALID BENNANI	FREE LANCE	pbtarchi@gmail.com	2026-08-01	Exp
N°9111	ZINEB CHRAIBI	AIR ARABIA	tarchioussama4@gmail.com	2026-07-31	Exp
N°9112	KARIM SAAD	ROYAL AIR MAROC	tarchioussama0974.fsts@uhp.ac.ma	2026-08-02	Exp
N°9113	FATIMA ZAHRA NADIR	SWISS PORT	oussamatarchi35@gmail.com	2026-07-31	Exp
N°9114	HAMZA EL AMRAI	ONDA DAKHLA	pbtarchi@gmail.com	2026-07-31	Exp
N°9115	SARA CHAOUI	STAG	tarchioussama157@gmail.com	2026-07-29	Exp

FIGURE 5.3 – Test de l'interface principale

Les filtres permettent de passer rapidement d'une vue globale à une vue centrée sur les badges expirés ou les badges à relancer. Cette fonctionnalité facilite le travail de l'utilisateur, car les cas urgents sont immédiatement visibles.

5.7 Tests des notifications locales

Les notifications locales ont été testées avec des badges expirés et des badges proches de l'expiration. Lorsqu'un badge nécessitant une attention est détecté, l'application affiche une fenêtre d'alerte contenant la liste des badges concernés.

La figure 5.5 présente un exemple de notification locale affichée par l'application lorsqu'elle détecte des badges expirés ou des badges proches de leur date d'expiration. Cette alerte permet d'attirer immédiatement l'attention de l'utilisateur sans qu'il soit obligé de consulter manuellement le fichier Excel.

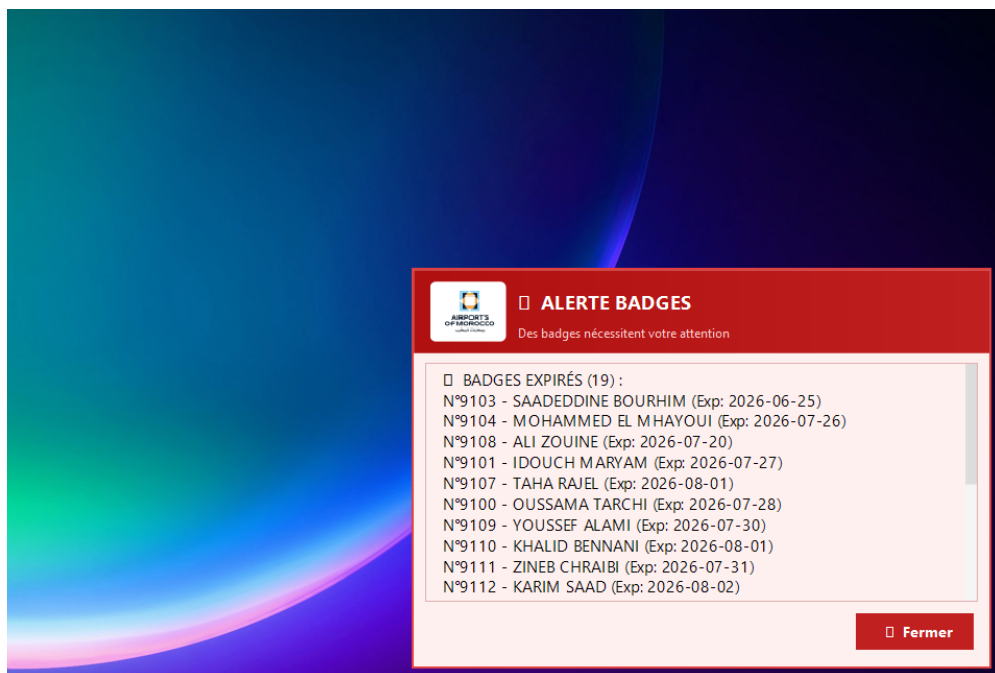


FIGURE 5.4 – Affichage d'une notification locale pour les badges nécessitant une attention

Le test a également porté sur la mémorisation des badges déjà vus. Une fois une alerte affichée, l'application enregistre l'information localement afin d'éviter de répéter inutilement la même notification.

```

{} sent_emails_history.json X {} seen_badges.json X
onda-badges-dakhla > desktop-app > {} seen_badges.json > ...

1
2 "N°9117|2026-07-26",
3 "N°9128|2026-08-11",
4 "N°9120|2026-07-25",
5 "N°9103|2026-06-25",
6 "N°9107|2026-08-01",
7 "N°9113|2026-07-31",
8 "N°9109|2026-07-30",
9 "N°9111|2026-07-31",
10 "N°9101|2026-07-27",
11 "N°9114|2026-07-31",
12 "N°9116|2026-07-28",
13 "N°9119|2026-08-04",
14 "N°9115|2026-07-29",
15 "N°9104|2026-07-26",
16 "N°9112|2026-08-02",
17 "N°9118|2026-07-31",
18 "N°9100|2026-07-28",
19 "N°9110|2026-08-01",
20 "N°9108|2026-07-20"
21

```

FIGURE 5.5 – fichier json

5.8 Tests d'envoi des e-mails

L'envoi des e-mails a été testé à partir des paramètres SMTP saisis dans l'interface. L'application prépare un message HTML contenant les informations du badge et l'envoie au détenteur concerné.



FIGURE 5.6 – Test d'envoi d'une notification par e-mail

Les tests ont confirmé que l'application peut générer un message clair indiquant le numéro du badge, le nom du détenteur, l'organisme et la date d'expiration. Cette notification permet d'informer rapidement la personne concernée et de faciliter le processus de renouvellement.

5.9 Tests de l'historique

L'historique constitue un élément important pour éviter les doublons. Les tests ont porté sur deux fichiers : celui des badges déjà vus et celui des e-mails déjà envoyés.

Lorsqu'un e-mail est envoyé, l'application enregistre les informations de l'opération dans l'historique. Avant un nouvel envoi, elle vérifie si une relance similaire a déjà été effectuée le même jour.

Test	Résultat attendu	État
Enregistrement d'un badge vu	Ajout dans <code>seen_badges.json</code>	Validé
Enregistrement d'un e-mail envoyé	Ajout dans <code>sent_emails_history.json</code>	Validé
Nouvel envoi le même jour	Envoi bloqué pour éviter le doublon	Validé
Consultation de l'historique	Affichage des actions enregistrées	Validé

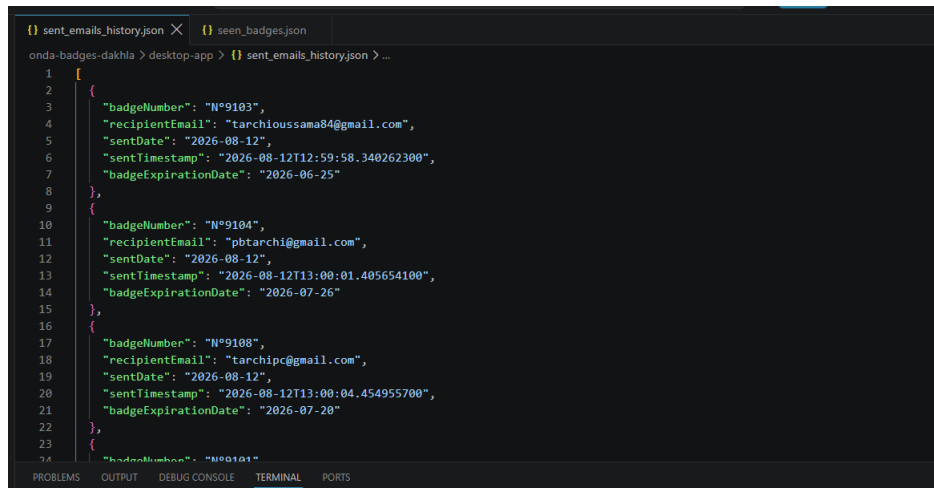
TABLE 5.4 – Tests de l'historique et de la prévention des doublons

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\user\Videos\projet de stage 2\onda-badges-dakhla\desktop-app> mvn compile exec:java
[INFO] --- resources:3.4.0:resources (default-resources) @ onda-badges-desktop ---
[INFO] Copying 2 resources from src\main\resources to target\classes
[INFO] --- compiler:3.15.0:compile (default-compile) @ onda-badges-desktop ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- exec:3.3.0:java (default-cli) @ onda-badges-desktop ---
Balayage automatique (Live-Refresh) démarré.
[Config] Configuration SMTP sauvegardée.
[EmailService] Email envoyé à tarchioussama84@gmail.com pour le badge N°9103
[EmailService] Email envoyé à pbtarchi@gmail.com pour le badge N°9104
[EmailService] Email envoyé à tarchipc@gmail.com pour le badge N°9108
[EmailService] Email envoyé à tarchi.oussama0974.fsts@uhp.ac.ma pour le badge N°9101
[EmailService] Email envoyé à xggdgdftwt@gmail.com pour le badge N°9107
[EmailService] Email envoyé à gdyddvtjff@gmail.com pour le badge N°9100
[EmailService] Email envoyé à earattzhdyzg@gmail.com pour le badge N°9102
[EmailService] Email envoyé à tarchi.oussama0974.fsts@uhp.ac.ma pour le badge N°9109
[EmailService] Email envoyé à pbtarchi@gmail.com pour le badge N°9110
[EmailService] Email envoyé à tarchioussama84@gmail.com pour le badge N°9111
[EmailService] Email envoyé à tarchi.oussama0974.fsts@uhp.ac.ma pour le badge N°9112
Terminate batch job (Y/N)?

```

FIGURE 5.7 – Test d'envoi d'une notification par e-mail



```

1  [
2    {
3      "badgeNumber": "N°9103",
4      "recipientEmail": "tarchioussama84@gmail.com",
5      "sentDate": "2026-08-12",
6      "sentTimestamp": "2026-08-12T12:59:58.340262300",
7      "badgeExpirationDate": "2026-06-25"
8    },
9    {
10     "badgeNumber": "N°9104",
11     "recipientEmail": "pbtarchi@gmail.com",
12     "sentDate": "2026-08-12",
13     "sentTimestamp": "2026-08-12T13:00:01.405654100",
14     "badgeExpirationDate": "2026-07-26"
15   },
16   {
17     "badgeNumber": "N°9108",
18     "recipientEmail": "tarchipc@gmail.com",
19     "sentDate": "2026-08-12",
20     "sentTimestamp": "2026-08-12T13:00:04.454955700",
21     "badgeExpirationDate": "2026-07-28"
22   },
23   {
24     "badgeNumber": "N°9101"

```

FIGURE 5.8 – Tests de l'historique des emails

5.10 Validation globale de la solution

Les tests réalisés montrent que l'application répond aux objectifs principaux du projet. Elle permet de remplacer une partie du suivi manuel par un traitement automatisé, tout en conservant le fichier Excel comme source principale des données.

La solution apporte plusieurs améliorations :

- gain de temps dans la vérification des badges ;
- diminution du risque d'oubli ou d'erreur de calcul ;
- meilleure visibilité sur les badges expirés ou proches de l'expiration ;
- relance plus rapide des détenteurs concernés ;
- conservation d'un historique simple des actions effectuées.

Ainsi, l'application constitue une solution adaptée au besoin exprimé par le service, tout en restant simple à déployer et à utiliser.

5.11 Limites de la solution

Malgré les résultats obtenus, certaines limites restent présentes. L'application dépend de la qualité du fichier Excel fourni. Si une colonne essentielle est absente, si une date est mal saisie ou si une adresse e-mail est incorrecte, le traitement peut être incomplet.

De plus, l'application fonctionne localement sur un poste de travail. Elle ne propose pas encore une gestion multi-utilisateur ni une base de données centralisée. La configuration SMTP est également saisie localement, ce qui peut être amélioré dans une version future.

5.12 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées afin d'enrichir l'application :

- ajouter une base de données pour centraliser les badges et les historiques ;
- mettre en place une authentification utilisateur ;
- améliorer la sécurité du stockage des paramètres SMTP ;
- générer des rapports PDF ou Excel sur les badges expirés ;
- ajouter des statistiques sur le nombre de badges valides, expirés ou à relancer ;
- prévoir une version web accessible depuis plusieurs postes ;
- intégrer une messagerie institutionnelle au lieu d'une configuration SMTP manuelle.

Ces perspectives permettraient de faire évoluer l'application vers une solution plus complète et mieux intégrée au système d'information de l'organisme.

5.13 Conclusion

Ce chapitre a présenté les tests réalisés pour valider le fonctionnement de l'application *ONDA Badges Dakhla*. Les résultats obtenus montrent que les fonctionnalités principales ont été mises en œuvre avec succès : lecture du fichier Excel, calcul des échéances, filtrage des badges, alertes locales, envoi des e-mails et gestion de l'historique.

La solution développée répond ainsi à la problématique initiale en automatisant le suivi des badges d'accès et en facilitant le travail du service concerné. Les limites identifiées ouvrent la voie à plusieurs perspectives d'amélioration pour les versions futures.

Conclusion générale

Ce stage effectué au sein de l'Office National Des Aéroports, à l'aéroport de Dakhla, m'a permis de travailler sur une problématique concrète liée au suivi des badges d'accès. Dans un environnement aéroportuaire, la gestion rigoureuse des autorisations d'accès représente un élément important pour la sûreté, l'organisation interne et la continuité des opérations.

Le projet *ONDA Badges Dakhla* avait pour objectif de concevoir et de développer une application desktop permettant d'automatiser une partie du suivi des badges. La solution réalisée permet de charger un fichier Excel, d'extraire les informations nécessaires, de calculer automatiquement les dates d'expiration, de classer les badges selon leur état, puis de déclencher des alertes locales ou des relances par courrier électronique.

La réalisation de cette application a permis de réduire les traitements manuels liés à la vérification des échéances. Elle offre également une meilleure visibilité sur les badges expirés ou proches de l'expiration, tout en conservant un historique des alertes et des e-mails envoyés afin d'éviter les répétitions inutiles. Grâce à son interface simple, l'application reste adaptée à une utilisation quotidienne par le service concerné.

Sur le plan technique, ce projet m'a permis de renforcer mes compétences en développement Java, en conception d'interfaces graphiques avec Swing, en lecture de fichiers Excel avec Apache POI, en envoi d'e-mails via SMTP et en gestion de fichiers locaux au format JSON. Il m'a également permis de mieux comprendre l'importance de l'analyse du besoin, de la structuration du code et de la validation progressive d'une solution logicielle.

Malgré les résultats obtenus, certaines améliorations restent possibles. Une future version pourrait intégrer une base de données, une authentification des utilisateurs, une meilleure sécurisation des paramètres SMTP, ou encore la génération automatique de rapports. Ces évolutions permettraient de rendre la solution plus complète et plus facilement intégrable dans un environnement professionnel plus large.

En conclusion, ce projet a constitué une expérience enrichissante à la fois sur le plan technique et professionnel. Il m'a permis d'appliquer mes connaissances académiques à un besoin réel, tout en contribuant à l'amélioration du suivi des badges d'accès au sein de l'aéroport de Dakhla.

Bibliographie

- [1] Office National Des Aéroports – ONDA.
Site officiel : <https://www.onda.ma>
- [2] Oracle.
Documentation officielle du langage Java.
<https://docs.oracle.com/en/java/>
- [3] Apache Maven Project.
Documentation officielle de Maven.
<https://maven.apache.org/guides/>
- [4] Apache POI.
Documentation officielle pour la manipulation des fichiers Microsoft Office.
<https://poi.apache.org/>
- [5] Jakarta Mail.
Documentation relative à l'envoi de courriers électroniques en Java.
<https://eclipse-ee4j.github.io/angus-mail/>
- [6] FlatLaf.
Documentation de la bibliothèque FlatLaf pour les interfaces Java Swing.
<https://www.formdev.com/flatlaf/>
- [7] Oracle.
Documentation Java Swing.
<https://docs.oracle.com/javase/tutorial/uiswing/>